

计算机程序设计基础(2)

--- C++程序设计(5)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 3.

4.5 特殊操作符的重载

- 上面举例介绍的几个操作符的重载，属于比较简单易于实现的。
- 还有一些运算符：像[]、=、类型转换等比较复杂，涉及到返回引用、模板等概念。这里先介绍=、类型转换等操作符的重载，在我们讲完模板后再介绍如何重载像[]等这些运算符。
- 先看一个例子：

```

#include <iostream>
#include <cstring>
using namespace std;
class String
{
public:
    String(const char * a);
    ~String( );
    String(const String &str2);
    void Set(const char *a);
    void Print( );
private:
    char *p;
};
String::String(const char *a)
{
    this->p=new char[strlen(a)+1];
    strcpy(p,a);
    cout<<"****构造函数被调用****"<<endl;
}

```

```

String::String(const String &str2)
{
    this->p=new char[strlen(str2.p) +1];
    strcpy(this->p, str2.p);
    cout<<"****复制构造函数被调用****"<<endl;
}
String::~~String()
{
    delete [ ]p;
    cout<<"****析构函数被调用****"<<endl;
}
void String::Set(const char *a)
{
    delete [ ]p;
    this->p=new char[strlen(a)+1];
    strcpy(this->p,a);
}
void String::Print( )
{
    cout<<"string: "<<p<<endl;
}

```

```
void main( )  
{ String str1("abcdefg"),str2("fgh"),str3("sssst");  
  str1.Print( );  
  str2.Print( );  
  str3.Print( );  
  str2=str1;  
  str2.Print( );  
  str2=str1=str3;  
  str2.Print( );  
  str3.Print( );  
}
```

语句 str2=str1;是对象之间赋值，不会触发String复制构造函数。

运行结果：

```
C:\Windows\system32\cmd.exe
****构造函数被调用****
****构造函数被调用****
****构造函数被调用****
string:  abcdefg
string:   fgh
string:  sssst
string:  abcdefg
string:  sssst
string:  sssst
****析构函数被调用****
```

程序运行到这里卡住不动了，出现了致命错误！因为 **str2 = str1;**是“浅复制”，导致两个对象的**p**指针指向同一个内存上的字符串，程序运行结束时，析构函数分别释放两个对象的**p**指针所致内存时产生致命错误。

如果想用 **=** 操作符赋值，就需要避免浅复制，重载**=**操作符。

把=操作符重载为String类的成员函数:

```
String& String::operator =(String str2)
{ delete [ ]this->p;
  this->p=new char[strlen(str2.p)+1];
  strcpy(this->p,str2.p);
  return *this;
}
```

在String类中增加:

```
String& operator =(String str2);
```

这里=操作符重载函数返回“当前对象String引用”的原因是，=操作符可能会出现连续赋值:

```
str3 = str2 = str1;
```

这等价于:

```
str3 = (str2 = str1);
```

运行结果:

```
****构造函数被调用****
****构造函数被调用****
****构造函数被调用****
string: abcdefg
string: fgh
string: sssst
****复制构造函数被调用****
****析构函数被调用****
string: abcdefg
****复制构造函数被调用****
****析构函数被调用****
****复制构造函数被调用****
****析构函数被调用****
string: sssst
string: sssst
****析构函数被调用****
****析构函数被调用****
****析构函数被调用****
请按任意键继续...
```



```
C:\Windows\system32\cm...
****构造函数被调用****
****构造函数被调用****
****构造函数被调用****
string: abcdefg
string: fgh
string: sssst
****复制构造函数被调用****
****析构函数被调用****
string: abcdefg
****复制构造函数被调用****
****析构函数被调用****
****复制构造函数被调用****
****析构函数被调用****
string: sssst
string: sssst
****析构函数被调用****
****析构函数被调用****
****析构函数被调用****
请按任意键继续...
```


能否改进提高执行效率？

能！把=操作符成员函数的参数改为引用方式：

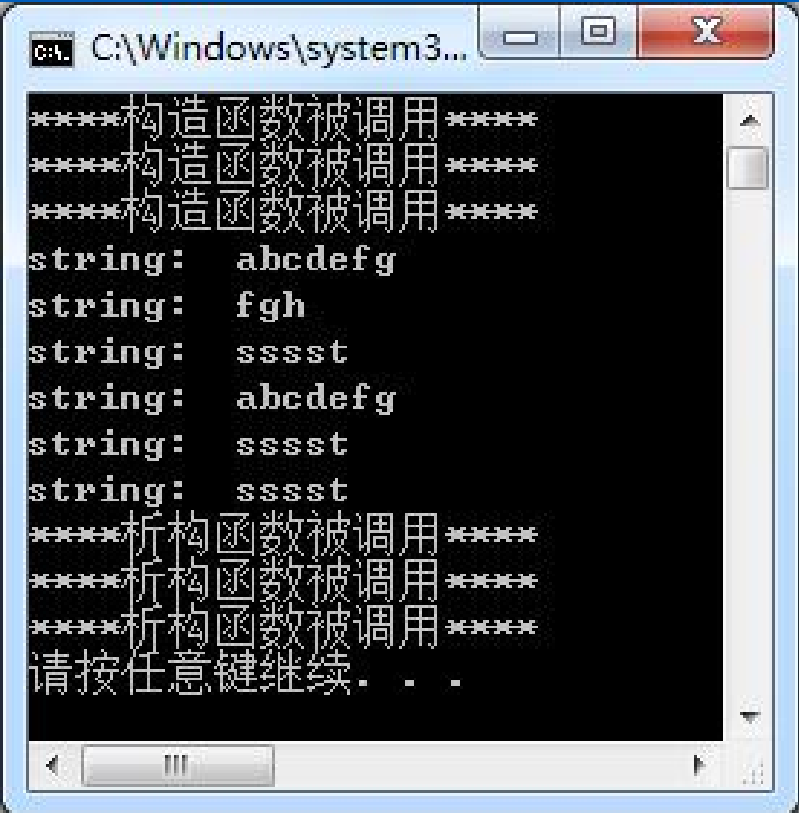
```
String& String::operator =(const String &str2)
```

```
{ delete [ ]p;  
  this->p=new char[strlen(str2.p)+1];  
  strcpy(this->p,str2.p);  
  return *this;  
}
```

运行结果：

```
*****构造函数被调用*****  
*****构造函数被调用*****  
*****构造函数被调用*****  
string: abcdefg  
string: fgh  
string: sssst  
string: abcdefg  
string: sssst  
string: sssst  
*****析构函数被调用*****  
*****析构函数被调用*****  
*****析构函数被调用*****  
请按任意键继续...
```

少了3次复制构造，同时也就少了3次析构。



```
C:\Windows\system32\cmd.exe  
*****构造函数被调用*****  
*****构造函数被调用*****  
*****构造函数被调用*****  
string: abcdefg  
string: fgh  
string: sssst  
string: abcdefg  
string: sssst  
string: sssst  
*****析构函数被调用*****  
*****析构函数被调用*****  
*****析构函数被调用*****  
请按任意键继续...
```

但如果程序中如果出现

```
str2=str2;
```

或者 (str2=str1)=str2;

将导致出现致命错误！原因是：此时=操作符重载函数中的str2和this指针所指的当前对象是同一个对象，delete []p;释放了当前对象的p所指动态内存，str2.p所指内存也就被释放了，此时执行strlen(str2.p)和strcpy(this->p,str2.p);必然引起致命错误。

所以需要进一步改写=操作符重载函数为：

```
String &String::operator =(String &str2)
{
    char *q=new char[strlen(str2.p)+1]; // 先申请新内存q
    strcpy(q, str2.p); // 把str2.p所指字符串复制过去
    delete [ ]this->p; // 删除当前对象中p所指内存
    this->p = q;       // 让当前对象中p指向新内存q
    return *this;
}
```

也可以把=操作符重载函数修改为:

```
String &String::operator =(const String &str2)
{ if (&str2 != this )
    { delete [ ]this->p; // 删除当前对象中p所指内存
      this->p =new char[strlen(str2.p)+1]; // 申请新内存
      strcpy(this->p, str2.p); // 把str2.p所指字符串复制过去
    }
  return *this;
}
```

判断如果str2和当前对象为同一个对象，则不做任何操作；否则再释放当前对象中p的动态内存，按照str2对象中字符串的长度申请一块新的动态内存，并复制字符串到当前对象中，从而实现“深复制”。

4.6 << 操作符的重载

- ◆ 操作符<<在C语言中的本意是位的左移，C++把此操作符重载为“流插入运算符”：

`cout <<i <<endl;`

进行输出，cout是ostream类的对象。

- ◆ 操作符>>在C语言中的本意是位的右移，C++把此操作符重载为“流提取运算符”：

`cin >>i;`

进行输入，cin是istream类的对象。

- ◆ 因此程序开头加上

`#include <iostream>`

是必须的，以定义cout、cin对象。

- ◆ 对 << 和 >> 重载的函数头部形式是：
ostream &operator << (ostream &, 自定义类型 &);
istream &operator >> (istream &, 自定义类型 &);
- ◆ 函数参数和返回值都必须是引用方式，而且只能将 <<和>> 重载为类的友元函数。
- ◆ 原因是，若想将 <<和>>重载为类的成员函数，则只能重载为ostream和istream类的成员函数，这就需要修改系统的ostream和istream类的定义，这是绝对不允许的！

◆ 例如把操作符<<重载输出复数complex类对象:

```
#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0,double i=0) : r(r), i(i) //构造函数
    { cout << "Constructor!" << *this; }
    // 用cout输出this指针所指的当前对象
    complex(const complex &c) : r(c.r),i(c.i) //复制构造函数
    {cout << "Copy Constructor!" << *this; }
    ~complex() { cout << "Destructor! " << *this; } // 析构
    friend complex operator + (complex &c1, complex &c2);
    friend complex operator - (complex &, complex &);
    friend ostream &operator << (ostream &, complex &);
private:         //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};
```

```

complex operator + (complex &c1,complex &c2)
{ return complex(c1.r+c2.r, c1.i+c2.i); }
complex operator - (complex &c1,complex &c2)
{ return complex(c1.r-c2.r, c2.i-c2.i); }
ostream & operator << (ostream &output, complex &c)
{ output << "(" << c.r << "," << c.i << ")" << endl;
  return output;
}
void main( ) //主函数
{ complex c1(5,4), c2(2,10), c3; //定义复数类的对象
  cout << "c1=" << c1;
    // 通过重载运算符<<把复数对象c1直接输出
  cout << "c2=" << c2;
  c3=c1-c2;      //使用重载运算符完成复数减法
  cout << "c3=c1-c2=" << c3;
  c3=c1+c2;      //使用重载运算符完成复数加法
  cout << "c3=c1+c2=" << c3;
}

```

运行结果:

Constructor! (5,4) // 构造c1

Constructor! (2,10) // 构造c2

Constructor! (0,0) // 构造c3

c1=(5,4)

c2=(2,10)

Constructor! (3,-6) // 构造一个无名对象返回

Destructor! (3,-6) // 赋值后立刻析构掉此无名对象

c3=c1-c2=(3,-6)

Constructor! (7,14) // 构造一个无名对象返回

Destructor! (7,14) // 赋值后立刻析构掉此无名对象

c3=c1+c2=(7,14)

Destructor! (7,14) // 析构c3

Destructor! (2,10) // 析构c2

Destructor! (5,4) // 析构c1

请按任意键继续...

- ◆ 通过重载<<运算符，使得看上去复数对象也可以像普通数据一样直接输出，减少了函数调用（不再需要单独调用Display()输出复数），简化了编程。
- ◆ 为什么重载<<运算符的函数头部必须写成：
ostream &operator << (ostream &output, complex &c)
呢？这是因为cout可以“级联”输出：
cout << c1 << c2 << c3;
上个语句自左至右执行，等价于：
((cout << c1) << c2) << c3;
- ◆ 执行cout << c1时，必须向系统的cout对象而不是向其副本对象输出c.r和c.i（因此形参output必须是引用），重载操作符函数<<必须返回系统cout对象的引用而不是其副本，为下一次输出做好准备，否则会引起混乱，导致下一次输出不成功。
- ◆ 强调两点：
 1. <<和>>只能重载为操作符友元函数。
 2. 重载函数的形参和返回值必须是I/O对象的引用。

4.7 类型操作符的重载---类型转换函数

- ◆ C++中不同数据类型之间，遵循向精度高类型看齐的原则，自动进行类型转换：

```
int k=5; double d; float f;  
d=7.5+k;
```

- ◆ 执行时，先把k的值转换成double型，再去求和，并把double类型的结果赋值给f。但如果是：

```
f=7.5+k;
```

编译时会出现：

warning C4244: “=”: 从“double”转换到“float”，可能丢失数据
为了避免出现警告信息，可以进行强制类型转换：

```
f=(float)(7.5+k); // C的书写方式，强制类型转换
```

```
f=float(7.5+k); // C++的书写方式，强制类型转换
```

```
f=float(7.5)+k; // float(7.5)把double数强制转换成float
```

- ◆ 如果有程序段：`complex c1(1,2), c2;`
`c2=c1+2.5;`

- ◆ 编译会出错，能否用`complex`进行强制类型转换：
`c2=c1+ complex(2.5);`

把2.5转换成复数对象`2.5+0i`，从而完成对象相加呢？

- ◆ 如果写成：`c2=c1+ complex(2.5, 0);`没有问题，把`c1`和无名对象`(2.5, 0)`求和赋值给`c2`。
- ◆ 如果写成`complex(2.5)`的形式，对于有参数缺省值的`complex`构造函数，也没有问题，这其实不是强制类型转换，而是构造一个无名对象，只是看上去跟`float(7.5)`的书写形式比较像，可以看作是强制类型转换。
- ◆ 如果`complex`构造函数参数没有缺省值，这需要写一个单个形参的构造函数`complex(double r)`：

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r, double i) : r(r), i(i) //构造函数
    { cout << "Constructor! " << *this; }
    complex(double r) : r(r), i(0)           // 转换构造函数
    { cout << "Cast Constructor! " << *this; }
    complex(const complex &c) : r(c.r), i(c.i) //复制构造函数
    { cout << "Copy Constructor!" << *this; }
    ~complex() { cout << "Destructor! " << *this; } // 析构
    friend complex operator + (complex &, complex &);
    friend complex operator - (complex &, complex &);
    friend ostream &operator << (ostream &, complex &);
private:         //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};

```

```

complex operator + (complex &c1,complex &c2)
{ return complex(c1.r+c2.r, c1.i+c2.i); }
complex operator - (complex &c1,complex &c2)
{ return complex(c1.r-c2.r, c2.i-c2.i); }
ostream & operator << (ostream &output, complex &c)
{output << "(" << c.r << ", " << c.i << ")" << endl;
 return output;
}
void main( ) //主函数
{ complex c1(5,4), c2(2,10), c3(0,0); // c3必须赋初值
  cout << "c1=" << c1;
  c3=c1+complex(2.5);//使用转换构造函数构造无名对象
  cout << "c3=c1+2.5=" << c3;
  c3=c1+c2+complex(3,4); //使用构造函数构造无名对象
  cout << "c3=c1+c2+(3,4)=" << c3;
}

```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
Cast Constructor! (2.5,0)
Constructor! (7.5,4)
Destructor! (7.5,4)
Destructor! (2.5,0)
c3=c1+2.5=(7.5,4)
Constructor! (3,4)
Constructor! (7,14)
Constructor! (10,18)
Destructor! (10,18)
Destructor! (7,14)
Destructor! (3,4)
c3=c1+c2+(3,4)=(10,18)
Destructor! (10,18)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

注意: 其中的构造/析构顺序

// 构造对象c1

// 构造对象c2

// 构造对象c3

// 构造(2.5, 0)

// 构造无名对象(7.5, 4)返回结果

// 赋值后析构无名对象(7.5, 4)

// 赋值后析构(2.5, 0)

// 构造无名对象(3, 4)

// 构造无名对象(7,14)返回c1+c2结果

// 构造无名对象(10,18)返回总的结果

// 赋值后析构无名对象(10, 18)

// 赋值后析构无名对象(7, 14)

// 赋值后析构无名对象(3, 4)

// 析构对象c3

// 析构对象c2

// 析构对象c1

类型转换函数

- ◆ 如果想把复数`complex`类对象`c`的实部当做普通`double`数进行运算，则需要`double(c)`---类型转换函数，把一个类的对象转换成另一个类型的数据。
 - ◆ **类型转换函数**的一般形式是：
operator 类型名()
{ return 与类型名类型相同的转换结果; }
 - ◆ 注意：
 1. 此函数前没有返回类型，函数也没有参数。函数将会自动返回与类型转换函数类型名相同的类型。
 2. 这里的类型名是基本数据类型`int`、`char`、`double`等。
 3. 类型转换函数只能作为相应类的成员函数。
- 例如：若要在复数`complex`类中实现`double(c)`:


```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r, double i) : r(r), i(i) //构造函数
    { cout << "Constructor! " << *this; }
    complex(double r) : r(r), i(0)           // 转换构造函数
    { cout << "Cast Constructor! " << *this; }
    complex(const complex &c) : r(c.r), i(c.i) // 复制构造函数
    { cout << "Copy Constructor!" << *this; }
    ~complex() { cout << "Destructor! " << *this; } // 析构函数
    operator double() //类型转换函数
    { cout << "Cast Double! " << r << endl; return r; }
    // 类型转换函数double返回当前对象的实部
    friend complex operator + (complex &, complex &);
    friend ostream &operator << (ostream &, complex &);
private:         //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};

```



```

complex operator + (complex &c1,complex &c2)
{ return complex(c1.r+c2.r, c1.i+c2.i); }
ostream & operator << (ostream &output, complex &c)
{ output << "(" << c.r << "," << c.i << ")" << endl;
  return output;
}
void main( ) //主函数
{ complex c1(5,4), c2(2,10); double k;
  c2=c1+complex(2.5); //使用转换构造函数
  cout << "c2=c1+2.5=" << c2;
  c2=c1+2.5; //使用类型转换函数
  cout << "c2=c1+2.5=" << c2;
  k = 2.5 + c2; // +满足交换律
  cout << "k = 2.5 + c2 = " << k << endl;
  k = c1 + c2;
  cout << "k = c1 + c2 = " << k << endl;
}

```

运行结果:

Constructor! (5,4)
Constructor! (2,10)
Cast Double! 5
Cast Constructor! (2.5,0)
Cast Double! 2.5
Cast Constructor! (7.5,0)
Destructor! (7.5,0)
Destructor! (2.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 5
Cast Constructor! (7.5,0)
Destructor! (7.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 7.5
k = 2.5 + c2 = 10
Constructor! (12.5,4)
Cast Double! 12.5
Destructor! (12.5,4)
k = c1 + c2 = 12.5
Destructor! (7.5,0)
Destructor! (5,4)
请按任意键继续...

结果分析:

构造对象c1(5,4)
构造对象c2(2,10)
将对象c1(5,4)强制转换为double数5
将2.5强制转换为无名对象(2.5,0)
将无名对象(2.5,0)强制转换为double数2.5
5+2.5求和得7.5,将7.5强制转换为无名对象(7.5,0)
赋值后,析构掉无名对象(7.5,0)
赋值后,析构掉无名对象(2.5,0)
打印出c2=c1+comlex(2.5)的结果(7.5,0)
将对象c1(5,4)强制转换为double数5
5+2.5求和得7.5,将7.5强制转换为无名对象(7.5,0)
赋值后,析构掉无名对象(7.5,0)
打印出c2=c1+2.5的结果(7.5,0)
将对象c2(7.5,0)强制转换为double数7.5
打印k = 2.5 + c2求和值10
+函数构造无名对象(12.5,4)返回求和结果
将无名对象(12.5,4)强制转换为double数12.5
析构掉无名对象(12.5,4)
打印k = c1 + c2 求和值12.5
析构对象c2 (7.5,0)
析构对象c1 (5,4)

若去掉转换构造函数，把构造函数第二个参数设成缺省值：

```
#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r, double i=0) : r(r), i(i) //构造函数
    { cout << "Constructor! " << *this; }
    complex(const complex &c) : r(c.r), i(c.i) // 复制构造函数
    { cout << "Copy Constructor!" << *this; }
    ~complex() { cout << "Destructor! " << *this; } // 析构函数
    operator double()
    { cout << "Cast Double! " << r << endl; return r; }
    // 类型转换函数double返回当前对象的实部
    friend complex operator + (complex &, complex &);
    friend ostream &operator << (ostream &, complex &);
private:          //私有数据成员
    double r;     //复数实部
    double i;     //复数虚部
};
```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Cast Double! 5
Constructor! (2.5,0)
Cast Double! 2.5
Constructor! (7.5,0)
Destructor! (7.5,0)
Destructor! (2.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 5
Constructor! (7.5,0)
Destructor! (7.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 7.5
k = 2.5 + c2 = 10
Constructor! (12.5,4)
Cast Double! 12.5
Destructor! (12.5,4)
k = c1 + c2 = 12.5
Destructor! (7.5,0)
Destructor! (5,4)
请按任意键继续...
```

修改前运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Cast Double! 5
Cast Constructor! (2.5,0)
Cast Double! 2.5
Cast Constructor! (7.5,0)
Destructor! (7.5,0)
Destructor! (2.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 5
Cast Constructor! (7.5,0)
Destructor! (7.5,0)
c2=c1+2.5=(7.5,0)
Cast Double! 7.5
k = 2.5 + c2 = 10
Constructor! (12.5,4)
Cast Double! 12.5
Destructor! (12.5,4)
k = c1 + c2 = 12.5
Destructor! (7.5,0)
Destructor! (5,4)
请按任意键继续...
```

结论:

- ◆ 运行结果没有改变，说明可以去掉所谓转换构造函数，由缺省值构造函数替代。
- ◆ 总的来看，类中的类型转换函数 `double()` 优先级很高，导致类的对象在运算时，不按常规方式操作，而是先进行类型强制转换，导致结果不可琢磨，像 `c2=c1+2.5` 结果是 `(7.5,0)` 而不是 `(7.5,4)`，因此**类型转换函数要慎用！**

4.8 重载操作符执行时参数的传递

- ◆ 通过重载<<运算符，cout可以“级联”输出：

`cout << c1 << c2 << c3;`

上个语句自左至右执行，等价于：

`((cout << c1) << c2) << c3;`

- ◆ 实际上的`cout << c1 << c2 << c3;`是如何调用<<操作符重载函数并传递参数的呢？
- ◆ 我们通过一个例子来说明，我们去掉参数的引用方式，并通过传递时的复制构造函数调用来跟踪参数的传递顺序。

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0,double i=0) : r(r), i(i) //构造函数
    { cout << "Constructor! ";
      cout << "(" << r << "," << i << ")" << endl;
    }
    complex(const complex &c) : r(c.r),i(c.i) //复制构造函数
    { cout << "Copy Constructor! ";
      cout << "(" << c.r << "," << c.i << ")" << endl;
    }
    ~complex( ) //析构函数
    { cout << "Destructor! ";
      cout << "(" << r << "," << i << ")" << endl;
    }
    friend ostream &operator << (ostream &, complex);

```

```

private:    //私有数据成员
    double r;//复数实部
    double i;//复数虚部
};
ostream & operator << (ostream &output, complex c)
{output << "(" << c.r << "," << c.i << ")" << endl;
  return output;
}
void main( ) //主函数
{ complex c1(1,1), c2(2,2), c3(3,3); //定义复数类的对象
  cout << c1 << c2 << c3;
}

```

注意：构造函数、复制构造函数、析构函数中不能再用 `cout << *this`; 直接输出当前对象，原因是：由于<<重载的参数传递不再是引用方式，需要复制构造来传递参数对象，这样写会导致无限自递归调用，造成**死循环**！

运行结果是:

Constructor! (1,1)

Constructor! (2,2)

Constructor! (3,3)

Copy Constructor! (3,3)

Copy Constructor! (2,2)

Copy Constructor! (1,1)

(1,1)

Destructor! (1,1)

(2,2)

Destructor! (2,2)

(3,3)

Destructor! (3,3)

Destructor! (3,3)

Destructor! (2,2)

Destructor! (1,1)

请按任意键继续...

分析:

构造对象c1(1,1)

构造对象c2(2,2)

构造对象c3(3,3)

从右至左, 先复制构造c3(3,3)

再复制构造c2(2,2)

最后复制构造c1(1,1)

打印c1(1,1)

析构形参对象(1,1)

打印c2(2,2)

析构形参对象(2,2)

打印c3(3,3)

析构形参对象(3,3)

析构对象c3(3,3)

析构对象c2(2,2)

析构对象c1(1,1)

请按任意键继续...

结论:

- ◆ 执行cout的<<时自左至右输出，但传递参数自右至左。与C语言函数参数传递的顺序一样。
- ◆ 这是因为栈操作的原因，先入后出，传递参数从右至左，保证了执行<<时自左至右的顺序。
- ◆ 我们通过另一个重载+为成员函数实现复数求和 $c4=c1+c2+c3$ 的例子来进一步说明参数的传递，给大家加深印象。
- ◆ 注意其中函数返回的无名对象的生存周期。

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0, double i=0) : r(r), i(i)
    { cout << "Constructor! " << *this; }
    complex(const complex &c) : r(c.r), i(c.i)
    { cout << "Copy Constructor!" << *this; }
    ~complex( ) { cout << "Destructor! " << *this; }
    complex operator + (complex c2); //运算符+重载成员函数
    friend ostream &operator << (ostream &, complex &);
private:         //私有数据成员
    double r; //复数实部
    double i; //复数虚部
};

```

```
complex complex::operator +(complex c2)
```

```
    //重载+运算符函数实现
```

```
    { return complex(r+c2.r, i+c2.i); }
```

```
ostream & operator << (ostream &output, complex &c)
```

```
{ output << "(" << c.r << "," << c.i << ")" << endl;
```

```
    return output;
```

```
}
```

```
void main( ) //主函数
```

```
{ complex c1(1,1), c2(2,2),c3(5,5), c4;//声明复数类的对象
```

```
    cout<<"c1=" << c1;
```

```
    cout<<"c2=" << c2;
```

```
    cout<<"c3=" << c3;
```

```
    c4=c1+c2+c3; //使用重载运算符完成复数加法
```

```
    cout<<"c4=c1+c2+c3=" << c4;
```

```
}
```

Constructor! (1,1)
Constructor! (2,2)
Constructor! (5,5)
Constructor! (0,0)
c1=(1,1)
c2=(2,2)
c3=(5,5)
Copy Constructor!(5,5)
Copy Constructor!(2,2)
Constructor! (3,3)
Destructor! (2,2)
Constructor! (8,8)
Destructor! (5,5)
Destructor! (8,8)
Destructor! (3,3)
c4=c1+c2+c3=(8,8)
Destructor! (8,8)
Destructor! (5,5)
Destructor! (2,2)
Destructor! (1,1)
请按任意键继续...

构造对象c1(1,1)
构造对象c2(2,2)
构造对象c3(5,5)
构造对象c4(0,0)
打印c1=(1,1)
打印c2=(2,2)
打印c3=(5,5)
c4=c1+c2+c3自右至左先传递参数c3(5,5)
再传递参数c2(2, 2)
第1个+函数构造无名对象 (3,3)返回
第1个+函数析构掉形参 (2,2)
第2个+函数构造无名对象 (8,8)返回
第2个+函数析构掉形参 (5,5)
给c4赋值后析构掉无名对象 (8,8)
析构掉无名对象 (3,3)
打印c4=c1+c2+c3=(8,8)
析构对象c4(8,8)
析构对象c3(5,5)
析构对象c2(2,2)
析构对象c1(1,1)
请按任意键继续...

4.9 重载++,--操作符函数的返回值

- ◆ 通过重载++, --运算符, 可以对复数对象进行加1、减1操作, 但如果有:

`c3 = ++c1 + c2++;`

前置++和后置++操作符重载函数应该如何返回值?

- ◆ 我们通过一个例子来说明。
- ◆ 这里把前置++和后置++操作符重载为友元函数。

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0, double i=0) : r(r), i(i) //构造函数
    { cout << "Constructor! " << *this; }
    complex(const complex &c) : r(c.r), i(c.i) // 复制构造函数
    {cout << "Copy Constructor!" << *this; }
    ~complex( ) { cout << "Destructor! " << *this; } // 析构函数
    friend complex &operator ++(complex &);// 操作符友元函数
    friend complex &operator ++(complex &, int);
    friend complex operator + (complex &, complex &);
    friend ostream &operator << (ostream &, complex &);
private:         //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};

```

```

complex &operator ++(complex &c)
{ c.r++; c.i++;
  return c; // 返回所操作的c对象
}
complex &operator ++(complex &c, int k)
{ double r=c.r, i=c.i;
  c.r++; c.i++;
  return complex(r, i); // 返回一个未加1的临时无名对象
}
complex operator + (complex &c1, complex &c2)
{ return complex(c1.r+c2.r, c1.i+c2.i);
}
ostream &operator << (ostream &output, complex &c)
{ output << "(" << c.r << "," << c.i << ")" << endl;
  return output;
}

```



```
void main( ) //主函数
{ complex c1(5,4), c2(2,10),c3;
  cout << "c1=" << c1;
  cout << "c2=" << c2;
  c3= ++c1 + c2++;
  cout << "c3= ++c1 + c2++ =" << c3;
  cout << "c1=" << c1;
  cout << "c2=" << c2;
}
```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Constructor! (2,10) //后置++返回一个无名对象(2,10)
Destructor! (2,10)  // 析构无名对象(2,10)
Constructor! (8,15) // +操作符函数返回一个无名对象(8,15)
Destructor! (8,15)  // 给c3赋值后析构无名对象(8,15)
c3= ++c1 + c2++ =(8,15)
c1=(6,5)
c2=(3,11)
Destructor! (8,15)
Destructor! (3,11)
Destructor! (6,5)
请按任意键继续...
```

你会看到无名对象(2,10)很早就析构掉了，似乎并没有等++执行完，而且编译时有警告错误信息：

test.cpp(25) : warning C4172: 返回局部变量或临时变量的地址

是指后++函数返回局部变量或临时变量的地址很危险，因此应该去掉后++函数返回值的引用方式：

complex operator ++(complex &c, int k)
此时的运行结果将是：

.....

Constructor! (2,10)

Constructor! (8,15)

Destructor! (8,15)

Destructor! (2,10)

c3= ++c1 + c2++=(8,15)

.....

无名对象(2,10)的析构延迟到了++执行完返回无名对象(8,15)给c3赋值，并析构掉无名对象(8,15)之后，这样就正常了。

把前置++和后置++操作符重载为成员函数:

```
#include <iostream>
```

```
using namespace std;
```

```
class complex {    //复数类声明
```

```
public:    //外部接口
```

```
    complex(double r=0, double i=0) : r(r), i(i) //构造函数
```

```
{ cout << "Constructor! " << *this; }
```

```
    complex(const complex &c) : r(c.r), i(c.i) // 复制构造函数
```

```
{cout << "Copy Constructor!" << *this; }
```

```
    ~complex() { cout << "Destructor! " << *this; } // 析构函数
```

```
    complex &operator ++( ); // 前++操作符成员函数
```

```
    complex operator ++(int); // 后++的返回值不再用引用方式
```

```
    friend complex operator + (complex &, complex &);
```

```
    friend ostream &operator << (ostream &, complex &);
```

```
private:    //私有数据成员
```

```
    double r; //复数实部
```

```
    double i; //复数虚部
```

```
};
```

```

complex &complex::operator ++( )
{ r++; i++;
  return *this; // 返回当前对象
}
complex complex::operator ++(int)
{ double r1=r, i1=i;
  r++; i++;
  return complex(r1,i1); // 返回一个未加1的临时无名对象
}
complex operator + (complex &c1, complex &c2)
{ return complex(c1.r+c2.r, c1.i+c2.i);
}
ostream &operator << (ostream &output, complex &c)
{ output << "(" << c.r << "," << c.i << ")" << endl;
  return output;
}

```

```
void main( ) //主函数
{ complex c1(5,4), c2(2,10),c3;
  cout << "c1=" << c1;
  cout << "c2=" << c2;
  c3= ++c1 + c2++; //使用类型转换函数
  cout << "c3= ++c1 + c2++=" << c3;
  cout << "c1=" << c1;
  cout << "c2=" << c2;
}
```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Constructor! (2,10) //后置++返回一个无名对象(2,10)
Constructor! (8,15) // +函数返回一个无名对象(8,15)
Destructor! (8,15)  // 给c3赋值后，析构无名对象(8,15)
Destructor! (2,10)  // 析构无名对象(2,10)
c3= ++c1 + c2++=(8,15)
c1=(6,5)
c2=(3,11)
Destructor! (8,15)
Destructor! (3,11)
Destructor! (6,5)
请按任意键继续...
```

第6次作业:

见网络学堂第6次作业

2题必做, 1题选做